

Problem 1

Problem Statement: Student Marks Analyzer (LEVEL-1)

Scenario:

You are developing a small utility for a teacher to analyze student marks stored in an array.

Requirements

- Store student marks in an array.
- Calculate total and average marks using array methods.
- Display pass/fail result based on average.
- Use let/const appropriately.
- Use arrow functions for calculations.
- Display output using template literals.

Technical Constraints

- Must use array methods like reduce() and map().
- Use only ES6+ syntax.
- No external libraries.
- All logic must be inside modular JavaScript file.

Learning Outcome

- Understand array processing.
- Practice ES6 features.
- Learn modular coding structure.

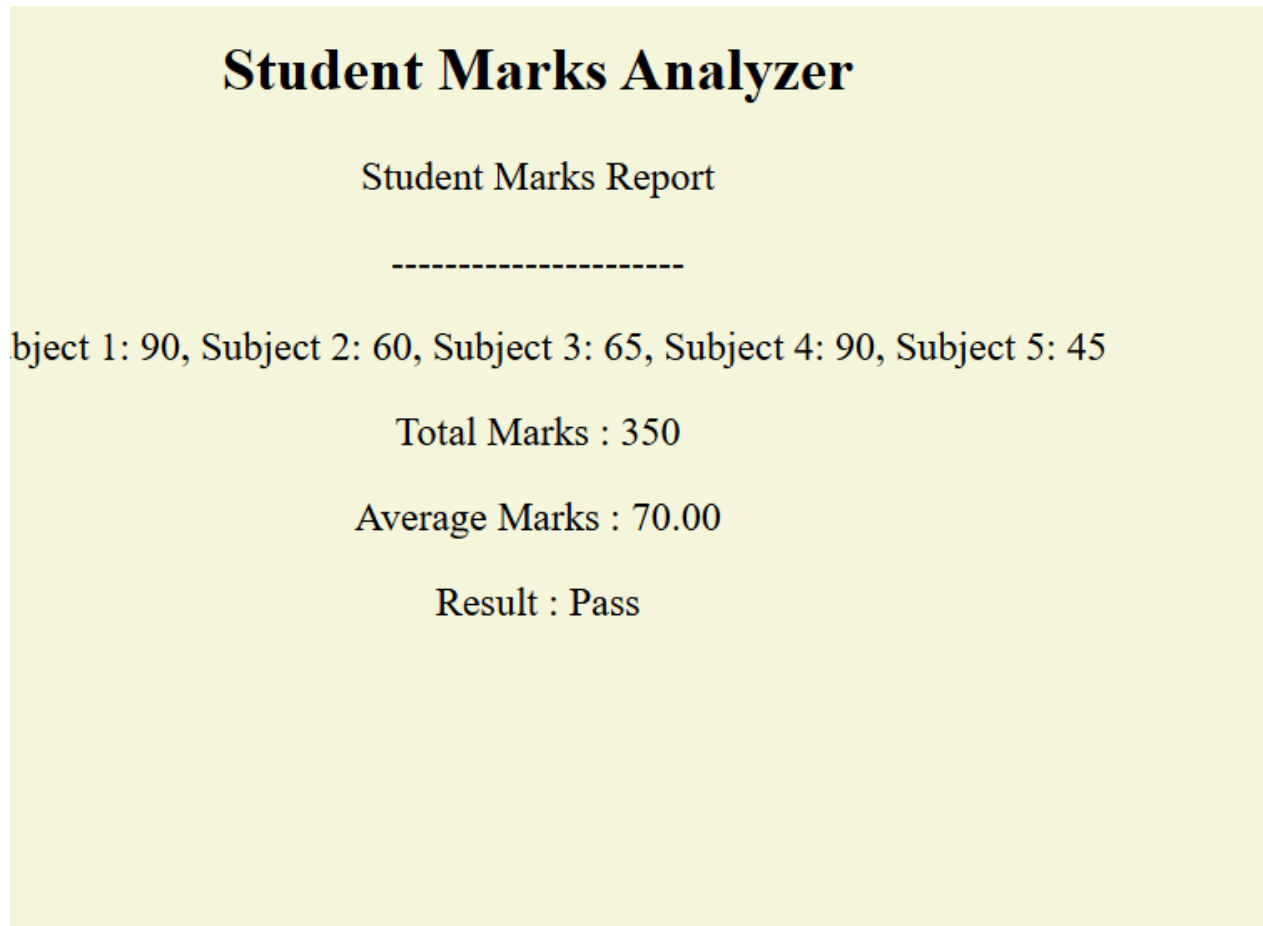
Source Code:

```
htmlform > <> studentmark.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Student Marks Analyzer</title>
7
8  <style>
9  body{
10     text-align: center;
11     background-color: #f5f5dc;
12  }
13 </style>
14
15 </head>
16
17 <body>
18
19 <h2>Student Marks Analyzer</h2>
20
21 <div id="output"></div>
22
23 <script src="script.js"></script>
24
25 </body>
26 </html>
```

```
htmlform > /* script.js > marks
1  const marks = [90, 60, 65, 90, 45];
2
3  const calculateTotal = (arr) => arr.reduce((sum, mark) => sum + mark, 0);
4
5  const calculateAverage = (arr) => {
6    const total = calculateTotal(arr);
7    return total / arr.length;
8  };
9
10 const getResult = (average) => (average >= 40 ? "Pass" : "Fail");
11
12 const formattedMarks = marks.map((mark, index) => `Subject ${index + 1}: ${mark}`);
13
14 const totalMarks = calculateTotal(marks);
15 const averageMarks = calculateAverage(marks);
16 const result = getResult(averageMarks);
17
18 console.log(`
19 Student Marks Report
20 -----
21 ${formattedMarks.join("\n")}
22
23 Total Marks      : ${totalMarks}
24 Average Marks    : ${averageMarks.toFixed(2)}
25 Result          : ${result}
26 `);
27
28 document.getElementById("output").innerHTML = `
29 <p>Student Marks Report</p>
30 <p>-----</p>
31 <p>${formattedMarks.join(", ")}</p>
32 <p>Total Marks : ${totalMarks}</p>
33 <p>Average Marks : ${averageMarks.toFixed(2)}</p>
34 <p>Result : ${result}</p>
35 `;
```

```
18 console.log(
19   Student Marks Report
20   -----
21   ${formattedMarks.join("\n")}
22
23   Total Marks      : ${totalMarks}
24   Average Marks    : ${averageMarks.toFixed(2)}
25   Result          : ${result}
26 `);
27
28 document.getElementById("output").innerHTML = `
29 <p>Student Marks Report</p>
30 <p>-----</p>
31 <p>${formattedMarks.join(", ")}</p>
32 <p>Total Marks : ${totalMarks}</p>
33 <p>Average Marks : ${averageMarks.toFixed(2)}</p>
34 <p>Result : ${result}</p>
35 `;
```

Output:



Explanation:

This code creates a **Student Marks Analyzer** that stores marks in an array and calculates the total and average using JavaScript functions. It also checks if the student passed or failed based on the average.

The output shows each subject mark, the total marks, the average marks, and the final result (Pass/Fail) displayed on the webpage.

Problem 2

Problem Statement: Product Cart Summary (LEVEL-1)

Scenario:

Build a simple shopping cart summary system.

Requirements

- Store product objects (name, price, quantity) in an array.
- Calculate total cart value.
- Display formatted invoice using template literals.
- Use arrow functions.
- Export calculation function from a module.

Technical Constraints

- Use map() and reduce().
- Use ES6 modules (export/import).
- No DOM manipulation required.

Learning Outcome

- Work with arrays of objects.
- Understand modules.

- Improve functional programming skills.

Source Code:

```
htmlform > <> productcart.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Product Cart Summary</title>
7
8    <style>
9      body{
10        text-align: center;
11        background-color: #f5f5dc;
12      }
13    </style>
14
15  </head>
16  <body>
17
18    <h2>Product Cart Summary</h2>
19
20    <script src="script.js"></script>
21
22  </body>
23  </html>
```

```
htmlform > js script.js > ...
1  const marks = [90, 60, 65, 90, 45];
2  const calculateTotal = (arr) => arr.reduce((sum, mark) => sum + mark, 0);
3  const calculateAverage = (arr) => {
4    const total = calculateTotal(arr);
5    return total / arr.length;
6  };
7  const getResult = (average) => (average >= 40 ? "Pass" : "Fail");
8  const formattedMarks = marks.map((mark, index) => `Subject ${index + 1}: ${mark}`);
9  const totalMarks = calculateTotal(marks);
10 const averageMarks = calculateAverage(marks);
11 const result = getResult(averageMarks);
12 console.log(`
13 Student Marks Report
14 -----
15 ${formattedMarks.join("\n")}
16 Total Marks      : ${totalMarks}
17 Average Marks    : ${averageMarks.toFixed(2)}
18 Result           : ${result}
19 `);
20 document.getElementById("output").innerHTML = `
21 <p>Student Marks Report</p>
22 <p>-----</p>
23 <p>${formattedMarks.join(", ")}</p>
24 <p>Total Marks : ${totalMarks}</p>
25 <p>Average Marks : ${averageMarks.toFixed(2)}</p>
26 <p>Result : ${result}</p>
27 `;
```

Output:



Output File: script.js

```
D:\upGrad\Training\Week2\Day2\Problem Statement 2>node script.js
```

```
----- INVOICE -----  
Laptop - ₹43750 x 1 = ₹43750  
Mouse - ₹500 x 1 = ₹500  
Keyboard - ₹1500 x 1 = ₹1500  
Laptop Bag - ₹1200 x 1 = ₹1200  
-----  
Total Amount: ₹46950  
-----
```

Explanation:

This code creates a **Product Cart Summary** using a list of products with price and quantity. JavaScript calculates the total cost using `map()` and `reduce()` and generates a simple invoice. The output shows each product with its price calculation and the **final total amount of ₹46950** printed in the console.

Problem 3

Problem Statement: **Weather Data Fetcher (LEVEL-2)**

Scenario:

Create an application that fetches weather data asynchronously.

Requirements

- **Fetch data from a public API using `fetch()`.**
- **Implement version using Promises.**
- **Implement version using `async/await`.**
- **Handle errors properly.**
- **Display formatted weather report.**

Technical Constraints -

Use `async/await`.

- **Use template literals.**
- **Use arrow functions.**
- **Must include proper error handling using `try/catch`.**

Learning Outcome

- **Understand Promises and `async/await`.**
- **Handle asynchronous operations.**
- **Improve API handling skills.**

Source code:

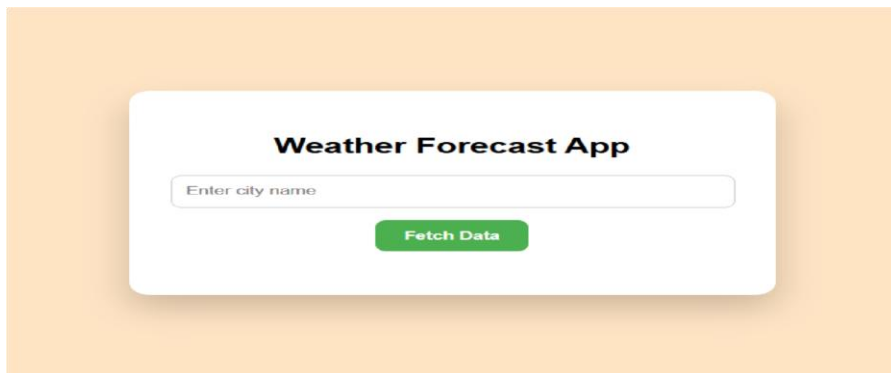
```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Weather Fetcher</title>
7  <style>
8  body {
9    font-family: Arial, sans-serif;
10   background: #1a202c;
11   display: flex;
12   justify-content: center;
13   align-items: center;
14   height: 100vh;
15   margin: 0;
16 }
17 .container {
18   background: white;
19   padding: 30px;
20   border-radius: 15px;
21   width: 400px;
22   text-align: center;
23   box-shadow: 0 15px 30px #000000;
24 }
25 input {
26   width: 95%;
27   padding: 10px;
28   margin-bottom: 15px;
29   border-radius: 8px;
30 }
31 button {
32   padding: 10px 20px;
33   border: none;
34   border-radius: 8px;
35   cursor: pointer;
36   font-weight: bold;
37   background-color: #4caf50;
38   color: white;
39 }
40 button:hover {
41   opacity: 0.9;
42 }
43 #weatherResult {
44   margin-top: 20px;
45   text-align: left;
46 }
47 </style>
48 </head>
49 <body>
50 <div class="container">
51 <h2>Weather Forecast App</h2>
52 <input type="text" id="cityInput" placeholder="Enter city name">
53 </div>
54 <button id="fetchBtn">Fetch Data</button>
55 <div id="weatherResult"></div>
56 </div>
57 <script>
58 const cityInput = document.getElementById("cityInput");
59 const resultDiv = document.getElementById("weatherResult");
60 const displayWeather = (data) => {
61   resultDiv.innerHTML = `
62     <h3>${data.location.name}, ${data.location.country}</h3>
63     <p><strong>🌡️ Temperature:</strong> ${data.current.temp_c}°C</p>
64     <p><strong>☁️ Condition:</strong> ${data.current.condition.text}</p>
65     <p><strong>💧 Humidity:</strong> ${data.current.humidity}%</p>
66     <p><strong>🌬️ Wind:</strong> ${data.current.wind_kph} kph</p>
67   `;
68 };
69 const fetchWeatherWithPromises = (searchCity) => {
70   const url = `https://api.weatherapi.com/v1/forecast.json?key=YOUR_API_KEY&q=${searchCity}`;
71   return fetch(url)
72     .then(response => {
73       if (!response.ok) {
74         throw new Error("City Not Found or API Error");
75       }
76       return response.json();
77     });
78 };
79 const fetchWeatherAsync = async (searchCity) => {
```

```

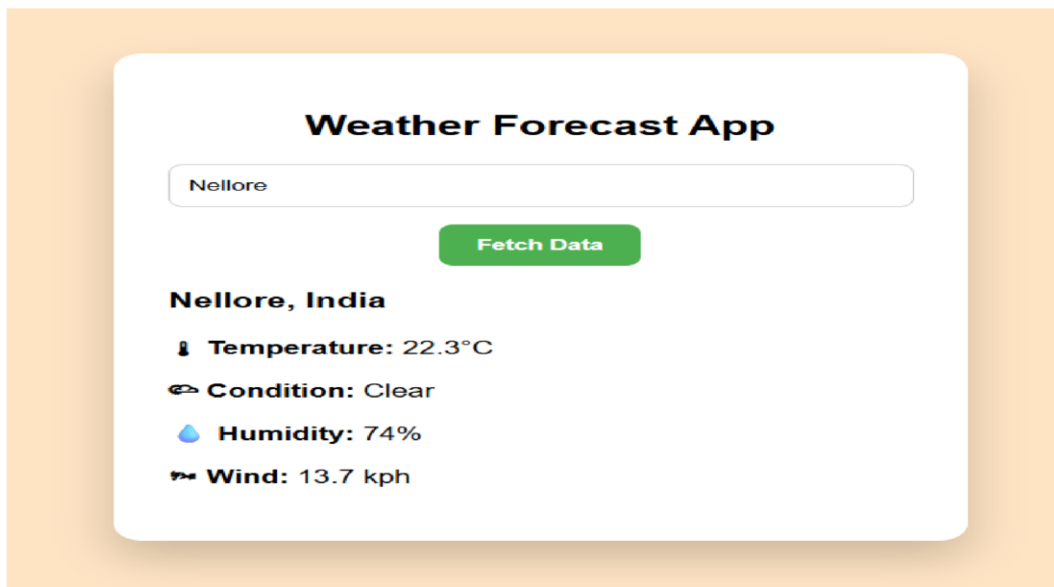
79 const fetchWeatherAsync = async (searchCity) => {
80   const response = await fetch(url);
81   if (!response.ok) {
82     throw new Error("City Not Found or API Error");
83   }
84   const data = await response.json();
85   displayWeather(data);
86 } catch (error) {
87   resultDiv.innerHTML = `Error: ${error.message}`;
88 }
89 };
90
91 document.getElementById("fetchBtn").addEventListener("click", () => {
92   const searchCity = cityInput.value.trim();
93   if (!searchCity) {
94     resultDiv.innerHTML = "Please Enter a City Name.";
95     return;
96   }
97   fetchWeatherAsync(searchCity);
98 });
99 </script>
100 </body>
101 </html>

```

Output:

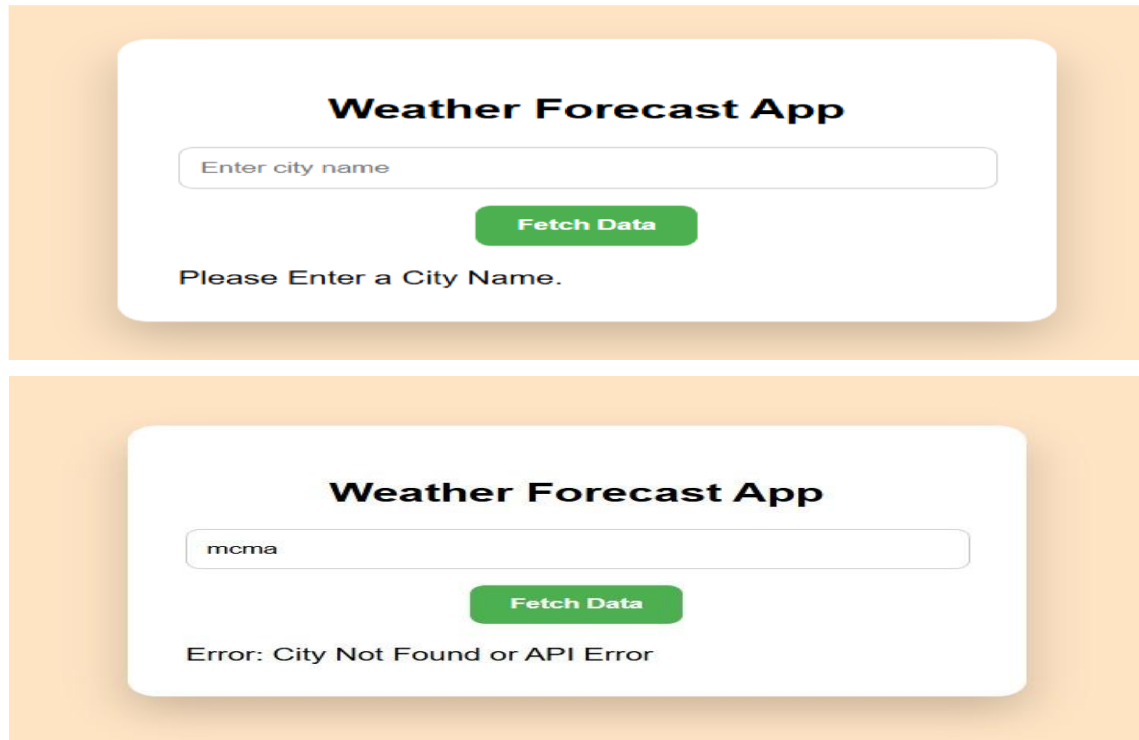


The image shows the initial state of the Weather Forecast App. It features a white card on an orange background. The card has a title "Weather Forecast App", a text input field with the placeholder "Enter city name", and a green "Fetch Data" button.



The image shows the Weather Forecast App after a successful fetch. The input field now contains "Nellore". Below the button, the following weather information is displayed:

- Nellore, India**
-  **Temperature:** 22.3°C
-  **Condition:** Clear
-  **Humidity:** 74%
-  **Wind:** 13.7 kph



Explanation:

This code creates a **Weather Forecast App** where the user enters a city name and clicks **Fetch Data** to get weather information from a public API. JavaScript uses **fetch()** with **async/await** to retrieve temperature, condition, humidity, and wind details and display them on the page. If the city is empty or invalid, it shows an **error message** like "Please Enter a City Name" or "City Not Found".

Problem 4

Problem Statement: **Task Manager with Async Storage (LEVEL-2)**

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

Requirements

- Store tasks in an array.
- Create `addTask()`, `deleteTask()`, and `listTasks()` functions.
- Simulate async storage using `setTimeout` with callbacks.
- Convert callback-based logic into Promises.
- Refactor into `async/await` version.
- Use ES6 modules.

Technical Constraints

- Must demonstrate `callback` → `promise` → `async/await` evolution.
- Use `let/const` properly.
- Use arrow functions.
- Use template literals for display.

Learning Outcome

- Deep understanding of async JavaScript.
- Learn refactoring async code.
- Master modern ES6+ features.

Source Code:

```

2    <html>
34   <body>
39   <script>
57   const listTasksPromise = () => {
58       return new Promise((resolve)=>{
60           resolve(tasks);
61       },1000);
62   });
63   };
64   const addTaskAsync = async () => {
65       const input = document.getElementById("taskInput")
66       const task = input.value;
67       if(task === ""){
68           alert("Enter a task");
69           return;
70       }
71       await addTaskPromise(task);
72       input.value="";
73       displayTasks();
74   };
75   const deleteTaskAsync = async (index) => {
76       await deleteTaskPromise(index);
77       displayTasks();
78   };
79   const displayTasks = async () => {
80
81       const taskList = document.getElementById("taskList
82
83       const tasks = await listTasksPromise();

```

```

39   <script>
64   const addTaskAsync = async () => {
74   };
75   const deleteTaskAsync = async (index) => {
76       await deleteTaskPromise(index);
77       displayTasks();
78   };
79   const displayTasks = async () => {
80
81       const taskList = document.getElementById("taskList");
82
83       const tasks = await listTasksPromise();
84
85       taskList.innerHTML="";
86
87       tasks.forEach((task,index)=>{
88
89           taskList.innerHTML += `
90               <li>
91                   ${task}
92                   <button class="deleteBtn" onclick="deleteTaskAsyn
93                   Delete
94                   </button>
95               </li>
96           `;
97       });
98   };

```

Task Manager

Add Task

Task Manager

Add Task

Task Manager

Add Task

Learn JavaScript

Delete

Explanation:

This code creates a simple **Task Manager** where a user can type a task and click **Add Task** to add it to the list. The tasks are stored in an array and displayed on the page, and each task has a **Delete button** to remove it. It uses **Promises and async/await with setTimeout** to simulate asynchronous operations like adding, listing, and deleting tasks before updating the UI.